

VideoRAG

Full-Stack RAG Chatbot — Complete Project Documentation

YouTube + Instagram • LangGraph • Qdrant • FastAPI • Next.js

Chapter 0 — Problem Statement (Original Assignment)

Assignment Rules

1. Do things all dynamically.
2. Must be a full-stack RAG chatbot.
3. Must use LangChain or LangGraph + embeddings + a vector DB. Use the stack listed — but if you have a better alternative with reasoning, that is welcome as an addition to the demo.
4. Every output must be dynamic / not hard-coded. Inputs can be hard-coded.
5. Once done, demo on a Loom; No bugs. No excuses.
 - Explain how this is the highest-quality, lowest-cost method to run this at scale (1000 creators a day). If it isn't the most efficient on cost and quality, tell us a better alternative.
 - Bottom line: The work must get done no matter what, in a timely manner (as per given deadline). Looking for REASONING on why this is the best solution.
 - We want you to think over costing & scalability.

Note

We don't want Claude prompts we want Engineers who can think & build scalable products.

RAG Chatbot Task

6. Take two social media video URLs as input (YouTube and Instagram Reels — mandatory).
7. Pull the transcript and metadata for both videos (views, likes, comments, creator, follower count, hashtags, upload date, duration).
8. Compute engagement rate = $(\text{likes} + \text{comments}) / \text{views} \times 100$.
9. Chunk + embed the transcripts and store in a vector DB (Pinecone, Weaviate, ChromaDB, Qdrant, or pgvector). Tag every chunk with video_id (A or B).
10. Build a RAG chat interface (LangChain or LangGraph) where a creator can ask:
 - Why did Video A get more engagement than Video B?
 - What's the engagement rate of each?
 - Compare the hooks in the first 5 seconds.
 - Who's the creator of Video B and what's their follower count?
 - Suggest improvements for B based on what worked in A.
11. Responses must stream, cite sources (which video + which chunk), and maintain memory across turns.
12. Build a vibe-coded frontend — side-by-side video cards + chat panel. We care about performance, speed & quality.

Required Tech Stack

Layer	Technology
Frontend	React / Next.js
Backend	FastAPI or Node.js
Orchestration	LangChain or LangGraph (mandatory)
Embeddings	OpenAI / Cohere / open-source (BGE, E5)
Vector DB	Pinecone / Weaviate / ChromaDB / Qdrant / pgvector
LLM	GPT-4o / Claude / Gemini / Llama
Transcript	youtube-transcript-api / yt-dlp + Whisper / AssemblyAI

Chapter 1 — Chosen Stack & Engineering Reasoning

Every decision below was made considering three constraints: speed to ship (30-hour deadline), quality of RAG output, and cost at scale (1000 creators/day). Each choice is defended against its alternatives.

Final Stack Decisions

Component	Choice	Key Reason
YouTube transcript	youtube-transcript-api	Free, instant (~0.3s), no AI needed, covers 90% of videos
YouTube metadata	YouTube Data API v3	Official, structured JSON, 10k free units/day = ~1000 videos
Instagram transcript	GROQ Whisper API	\$0.00185/min — 1-min reel costs <\$0.002, drops latency from 45s to under 1 second.
Instagram download	instaloader + yt-dlp fallback	instaloader for metadata (no login needed for public reels); yt-dlp as fallback if instaloader is rate-limited
Chunking	RecursiveCharacterTextSplitter 500/100	Balanced precision — not too small (loss of context) not too large (noise)
Embeddings	BAAI/bge-small-en-v1.5	\$0 forever, retrieval-optimised, 384-dim, ~2ms/chunk on CPU
Vector DB	Qdrant (Docker)	Rust-based speed, filter by video_id, free self-hosted, scales to millions
Orchestration	LangGraph	Parallel nodes, state graph, built-in checkpointer — not possible with plain LangChain
LLM	Groq llama-3.3-70b-versatile	128k context, sub-second TTFT, 14,400 free req/day, fastest inference available
Memory	LangGraph AsyncRedisSaver	Survives restarts, works across multiple workers, O(1) reads
Backend	FastAPI	Async-native, Pydantic, SSE trivial, fastest Python web framework
Frontend	Next.js App Router	SSR, API routes, best DX for React in 2024
Job state	Redis	Sub-ms reads, TTL built-in, pub/sub for WebSocket events

Why LangGraph Over Plain LangChain

Plain LangChain RAG is a straight line: retrieve then generate. You cannot run things in parallel, branch conditionally, or have background jobs. LangGraph gives you a graph with shared state — every node reads and writes to the same state object, enabling true parallelism.

Capability	LangChain	LangGraph
Parallel retrieval (A + B at once)	No	Yes — fan-out nodes
Conditional branching	Difficult	First-class with conditional edges
Memory across turns	Manual	Built-in checkpointer (Redis/memory)
State shared across steps	No	Yes — TypedDict state object
Streaming support	Partial	Native .astream() with stream_mode='messages'
Background job support	No	Yes — async nodes

Cost Analysis at Scale — 1000 Creators/Day

Operation	Choice	Cost per creator	1000 creators/day
YouTube transcript	youtube-transcript-api	\$0.00	\$0.00
YouTube metadata	YouTube Data API v3	\$0.00 (free quota)	\$0.00
Instagram transcript	Whisper API (1-min reel)	\$0.006	\$6.00
Embeddings (BGE-small local)	Self-hosted on CPU	\$0.00	\$0.00
Vector storage (Qdrant self-hosted)	Docker on your server	\$0.00	\$0.00
LLM (Groq free tier)	14,400 req/day free	\$0.00	\$0.00
LLM (Groq paid, if over limit)	\$0.59/M input tokens	~\$0.001/question	~\$1/day at 1000 Q
Redis (self-hosted)	Docker	\$0.00	\$0.00
TOTAL (typical day)		~\$0.007/creator	~\$7/day

Scale verdict

Running 1000 creators/day costs approximately \$7/day (\$210/month). The dominant cost is Whisper API for Instagram. If Instagram is dropped or replaced with free local Whisper (accepting the 45s latency), total cost is \$0/day. This beats any SaaS alternative by 10-100x.

Chapter 2 — System Architecture

2.1 The 7-Layer Mental Model

The system is built in 7 layers. Each layer has one job and passes a clearly defined data shape to the next.

Layer	Component	Input	Output
1. Ingestion	youtube-transcript-api, instaloader, Whisper API	URL string	Raw transcript + metadata JSON
2. Processing	Python logic	Raw JSON	Cleaned metadata + engagement_rate float
3. Chunking	RecursiveCharacterTextSplitter	Transcript string	List of {text, chunk_index, video_id, metadata}
4. Embedding	BAAI/bge-small-en-v1.5	Chunk text	float[384] vector per chunk
5. Storage	Qdrant	Vector + payload	Indexed points, filterable by video_id
6. Retrieval + RAG	LangGraph	User question	Top-k chunks from Qdrant, prompt assembled
7. Generation	Groq LLM (streaming)	Prompt + context	SSE token stream with citations

2.2 Parallelism Map — What Runs at the Same Time

Parallel Group	What runs together	Saving
Ingestion	YouTube fetch + Instagram fetch fired simultaneously with asyncio.gather()	~2s saved
Ingestion	YouTube transcript API + YouTube Data API v3 (both fired at once)	~0.3s saved
Chunking	Chunk Video A + Chunk Video B in parallel threads	~0.1s saved
Embedding	Embed Video A chunks + Video B chunks simultaneously (ThreadPoolExecutor)	~1s saved
Retrieval (every chat turn)	Qdrant search Video A + Qdrant search Video B (asyncio.gather)	~10ms saved per turn

2.3 Background vs Async vs Parallel — the Difference

Background: User gets HTTP response immediately (<50ms). Heavy work runs after. Example: POST /ingest returns job_id instantly, Whisper runs in background.

Async: Server doesn't block while waiting for I/O. Example: while waiting for Qdrant to respond, the server handles other requests. Uses async def + await.

Parallel: Two things run at literally the same clock time. Example: asyncio.gather(search_A(), search_B()) — both Qdrant searches fire simultaneously.

2.4 Data Schema — What Every Object Looks Like

Video metadata object

```
{
  "video_id": "A",
  "platform": "youtube",
  "url": "https://youtube.com/watch?v=...",
  "title": "How I grew 100k followers",
  "creator": "@mkbhd",
  "follower_count": 4200000,
  "views": 850000,
  "likes": 42000,
  "comments": 1800,
  "hashtags": ["#tech", "#youtube"],
  "upload_date": "2024-11-15",
  "duration_seconds": 187,
  "engagement_rate": 5.15,
  "transcript": "Hey guys, welcome back...",
}
```

Qdrant point (one per chunk)

```
{
  "id": "uuid-v4",
  "vector": [0.21, -0.54, 0.88, ...], // float[384]
  "payload": {
    "text": "The first thing I did was post every single day...",
    "video_id": "A",
    "chunk_index": 3,
    "platform": "youtube",
    "creator": "@mkbhd",
    "title": "How I grew 100k followers",
    "engagement_rate": 5.15,
    "views": 850000,
    "likes": 42000,
    "comments": 1800
  }
}
```


Chapter 3 — Ingestion Pipeline

3.1 YouTube — Transcript

Tool: `youtube-transcript-api` — Fetches pre-existing captions from YouTube servers. No AI. No audio processing.

Speed: ~0.3 seconds per video

Cost: \$0

Limitation: Only works if the video has captions enabled (>90% of videos do). For this project, we only support videos with captions — no fallback needed for the demo.

```
from youtube_transcript_api import YouTubeTranscriptApi

def get_youtube_transcript(video_id: str) -> str:
    segments = YouTubeTranscriptApi.get_transcript(video_id)
    # segments = [{text, start, duration}, ...]
    return ' '.join(s['text'] for s in segments)
```

3.2 YouTube — Metadata

Tool: YouTube Data API v3 (Google Cloud Console)

Free quota: 10,000 units/day. Each video lookup = 5 units. So 2,000 videos/day for free — covers 1000 creators with 2 videos each.

```
from googleapiclient.discovery import build

youtube = build('youtube', 'v3', developerKey=YOUTUBE_API_KEY)

def get_youtube_metadata(video_id: str) -> dict:
    res = youtube.videos().list(
        part='snippet,statistics,contentDetails',
        id=video_id
    ).execute()
    item = res['items'][0]
    stats = item['statistics']
    views = int(stats.get('viewCount', 0))
    likes = int(stats.get('likeCount', 0))
    comments = int(stats.get('commentCount', 0))
    return {
        'title': item['snippet']['title'],
        'creator': item['snippet']['channelTitle'],
        'views': views, 'likes': likes, 'comments': comments,
        'engagement_rate': round((likes + comments) / views * 100, 2) if views else 0,
        'upload_date': item['snippet']['publishedAt'][:10],
        'hashtags': item['snippet'].get('tags', []),
    }
```

3.3 Instagram — Transcript via Whisper API

Primary tool: instaloader (metadata + download) + GROQ Whisper API (transcription)
Fallback tool: yt-dlp — if instaloader is rate-limited, yt-dlp downloads the .mp4 without login
Speed: ~3-5 seconds total — dominated by Whisper API, not the download step
Cost: \$0.006 per minute of audio. A 1-minute reel costs \$0.006

Why NOT instagrapi (decision record)

instagrapi uses Instagram's private mobile API and is sometimes proposed as a faster alternative to instaloader. We evaluated it and decided against it for the following reasons:

Factor	instaloader (our choice)	instagrapi (rejected)
Login required	No — works on public reels without credentials	Yes — requires a real Instagram account + session file
Bottleneck relevance	Download is NOT the bottleneck — Whisper API is (~3-5s regardless)	Speed advantage is on metadata fetch only, which is already <1s
Demo risk	Low — no auth state to expire mid-demo	High — session tokens expire, 2FA challenges can block the demo
Ban risk	Low for occasional use	Higher — private API mimicry is more aggressively blocked
Async support	Sync (runs in thread pool, fine for our use case)	Yes, but the async benefit is irrelevant when Whisper dominates latency
Conclusion	✓ Sufficient for this use case	X Adds complexity + risk with no meaningful speed gain

Key insight

The 10-50x speed claim for instagrapi applies to metadata scraping in isolation. Our actual bottleneck is Whisper API transcription (~3-5s). Switching from instaloader to instagrapi would save at most 0.5-1s on the download while adding login complexity and session management risk. Not worth it for Phase 1.

Why yt-dlp as fallback

instaloader occasionally gets rate-limited on public reels during development (HTTP 429). yt-dlp is a separate tool with different request patterns — having it as a fallback means a rate-limit on one path doesn't kill the demo.

```
import instaloader, GROQ, subprocess, tempfile, os
```

```

def download_reel_instaloader(shortcode: str, tmpdir: str) -> tuple[str, object]:
    '''Primary path - instaloader fetches metadata + video together.'''
    L = instaloader.Instaloader(download_pictures=False, quiet=True)
    post = instaloader.Post.from_shortcode(L.context, shortcode)
    L.download_post(post, target=tmpdir)
    mp4 = next(f for f in os.listdir(tmpdir) if f.endswith('.mp4'))
    return os.path.join(tmpdir, mp4), post

def download_reel_ytdlp(url: str, tmpdir: str) -> str:
    '''Fallback path - yt-dlp if instaloader is rate-limited.'''
    out_path = os.path.join(tmpdir, 'reel.mp4')
    subprocess.run([
        'yt-dlp', '-o', out_path, '--quiet',
        '--no-playlist', url
    ], check=True)
    return out_path

async def get_instagram_transcript(reel_url: str) -> dict:
    shortcode = reel_url.rstrip('/').split('/')[-1]
    client = openai.OpenAI()
    with tempfile.TemporaryDirectory() as tmpdir:
        try:
            mp4_path, post = download_reel_instaloader(shortcode, tmpdir)
            metadata = {
                'creator': f'@{post.owner_username}',
                'follower_count': post.owner_profile.followers,
                'likes': post.likes,
                'views': post.video_view_count or 0,
                'comments': post.comments,
                'upload_date': str(post.date.date()),
                'hashtags': list(post.caption_hashtags),
            }
        except Exception as e:
            print(f'instaloader failed ({e}), falling back to yt-dlp...')
            mp4_path = download_reel_ytdlp(reel_url, tmpdir)
            metadata = {'creator': 'unknown', 'likes': 0, 'views': 0}
            with open(mp4_path, 'rb') as f:
                result = client.audio.transcriptions.create(model='whisper-1', file=f)
    return {'transcript': result.text, **metadata}

```

3.4 Chunking Strategy

chunk_size = 500 characters, overlap = 100 characters

Why 500 chars? ~80-100 words. Enough context to answer a question meaningfully, small enough to be precise.

Why overlap? A sentence that falls at a chunk boundary doesn't get lost. The overlap of 100 chars ensures continuity between adjacent chunks.

Why RecursiveCharacterTextSplitter? Tries to split at paragraphs first, then sentences, then words, then characters — never cuts a word in half. Built into LangChain, zero cost.

```

from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(

```

```

        chunk_size=500,
        chunk_overlap=100,
        length_function=len
    )

def chunk_transcript(transcript: str, metadata: dict) -> list:
    chunks = splitter.split_text(transcript)
    return [
        {
            'text': chunk,
            'chunk_index': i,
            **metadata # video_id, creator, engagement_rate, etc.
        }
        for i, chunk in enumerate(chunks)
    ]

```

3.5 Embedding

Model: BAAI/bge-small-en-v1.5 — Local, free, 384-dimensional, retrieval-optimised

```

from sentence_transformers import SentenceTransformer

embedder = SentenceTransformer('BAAI/bge-small-en-v1.5')

def embed_chunks(chunks: list) -> list:
    texts = [c['text'] for c in chunks]
    vectors = embedder.encode(texts, normalize_embeddings=True)
    return [
        {'vector': vec.tolist(), **chunk}
        for vec, chunk in zip(vectors, chunks)
    ]

# CRITICAL: Use the same model to embed the user's question at query time

```

Critical rule

The question and the chunks **MUST** be embedded with the same model. BGE-small at ingestion + OpenAI at query time = vectors in different mathematical spaces = garbage retrieval results.

3.6 Qdrant Storage

```

from qdrant_client import AsyncQdrantClient
from qdrant_client.models import VectorParams, Distance, PointStruct
import uuid

qdrant = AsyncQdrantClient(host='localhost', port=6333)

async def setup_collection():
    await qdrant.recreate_collection(
        collection_name='videos',
        vectors_config=VectorParams(size=384, distance=Distance.COSINE)
    )

```

```
async def upsert_chunks(embedded_chunks: list):
    points = [
        PointStruct(
            id=str(uuid.uuid4()),
            vector=chunk['vector'],
            payload={k: v for k, v in chunk.items() if k != 'vector'}
        )
        for chunk in embedded_chunks
    ]
    await qdrant.upsert(collection_name='videos', points=points)
```

Chapter 4 — LangGraph RAG Pipeline

4.1 State Object — The Shared Memory

Every LangGraph node reads from and writes to a single TypedDict state object. This is what allows nodes to share data without passing arguments.

```
from typing import TypedDict, List, Optional

class RAGState(TypedDict):
    question: str
    job_id: str
    question_vector: Optional[List[float]]
    context_a: Optional[List[dict]]
    context_b: Optional[List[dict]]
    memory: Optional[List[dict]]
    answer: Optional[str]
    sources: Optional[List[dict]]
```

4.2 Graph Nodes

Node 1: load_memory

```
async def load_memory_node(state: RAGState) -> dict:
    # LangGraph checkpointer handles this automatically via thread_id
    # We just return the last N turns to keep within context window
    history = state.get('memory', [])
    return {'memory': history[-6:]} # sliding window of 6 turns
```

Node 2: embed_question

```
async def embed_question_node(state: RAGState) -> dict:
    # Run in thread pool so CPU work doesn't block event loop
    loop = asyncio.get_event_loop()
    vector = await loop.run_in_executor(
        None, lambda: embedder.encode([state['question']],
        normalize_embeddings=True)[0].tolist()
    )
    return {'question_vector': vector}
```

Node 3: retrieve_parallel

```
async def retrieve_parallel_node(state: RAGState) -> dict:
    # Both Qdrant searches fire at exactly the same time
    from qdrant_client.models import Filter, FieldCondition, MatchValue

    async def search(video_id: str):
        return await qdrant.search(
            collection_name='videos',
            query_vector=state['question_vector'],
            query_filter=Filter(must=[
                FieldCondition(key='video_id', match=MatchValue(value=video_id))
            ]),
            limit=4
        )
```

```

    )

    results_a, results_b = await asyncio.gather(search('A'), search('B'))
    return {'context_a': results_a, 'context_b': results_b}

```

Node 4: generate (streaming)

```

async def generate_node(state: RAGState) -> dict:
    chunks_a = format_chunks(state['context_a'], 'A')
    chunks_b = format_chunks(state['context_b'], 'B')
    history = format_history(state['memory'])

    prompt = f'''You are a video analytics expert.

    VIDEO A CONTEXT (cite as [Video A, chunk N]):
    {chunks_a}

    VIDEO B CONTEXT (cite as [Video B, chunk N]):
    {chunks_b}

    CONVERSATION HISTORY:
    {history}

    QUESTION: {state['question']]

    Rules: Always cite source chunks. Use engagement rates from metadata.
    Compare both videos when relevant. Be specific and actionable.
    '''

    # Streaming is handled at the FastAPI route level via .astream()
    return {'answer': prompt} # graph yields tokens via stream_mode='messages'

```

4.3 Graph Assembly

```

from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.redis.aio import AsyncRedisSaver

graph_builder = StateGraph(RAGState)
graph_builder.add_node('load_memory', load_memory_node)
graph_builder.add_node('embed_question', embed_question_node)
graph_builder.add_node('retrieve', retrieve_parallel_node)
graph_builder.add_node('generate', generate_node)

graph_builder.add_edge(START, 'load_memory')
graph_builder.add_edge('load_memory', 'embed_question')
graph_builder.add_edge('embed_question', 'retrieve')
graph_builder.add_edge('retrieve', 'generate')
graph_builder.add_edge('generate', END)

# Redis checkpointer - memory survives restarts and scales across workers
async def create_graph():
    saver = AsyncRedisSaver.from_conn_string('redis://localhost:6379')
    return graph_builder.compile(checkpointer=saver)

```

4.4 Context Window Management

Every LLM call must fit within the model's context window. With Groq llama-3.3-70b (128k tokens), this is not a concern for normal usage, but the budget should always be tracked.

Component	Estimated tokens	Notes
System prompt	~300	Fixed, written once
Conversation history (6 turns)	~1200	~200 tokens/turn x 6
Retrieved chunks A (4 chunks x 120 tokens)	~480	Filtered by video_id
Retrieved chunks B (4 chunks x 120 tokens)	~480	Filtered by video_id
User question	~30	Typical question
TOTAL INPUT	~2490	Well within 128k limit
LLM response	~400-600	Streamed back
TOTAL PER TURN	~3000-3100	~\$0.001 at Groq pricing

History summarisation (when history exceeds 1500 tokens)

```
async def maybe_summarise_history(history: list) -> list:
    token_count = sum(count_tokens(turn['content']) for turn in history)
    if token_count > 1500:
        summary_prompt = f'Summarise in 2 sentences: {history}'
        summary = await llm.ainvoke(summary_prompt)
        return [{'role': 'system', 'content': f'Prior context: {summary.content}'}]
    return history
```


Chapter 5 — FastAPI Backend

5.1 File Structure

```
backend/
├── main.py           # app init, CORS, route mounting
├── routes/
│   ├── ingest.py    # POST /ingest
│   ├── status.py    # GET /status/{job_id}
│   ├── chat.py      # POST /chat (SSE streaming)
│   └── ws.py        # WebSocket /ws/{job_id}
├── services/
│   ├── ingestion.py # YouTube + Instagram pipeline
│   ├── embedder.py  # BGE-small wrapper (singleton)
│   ├── vector_store.py # Qdrant async client wrapper
│   └── llm.py        # LangGraph graph definition
├── models.py        # Pydantic request/response schemas
└── state.py         # Redis job state manager
```

5.2 API Routes — The Full Contract

Route	Method	Body / Params	Response	Notes
POST /ingest	POST	{yt_url, ig_url}	{job_id, status: 'queued'}	Returns in <50ms, fires background job
GET /status/{job_id}	GET	job_id in URL	{pct, stage, video_ready: ['A','B']}	Frontend polls every 2s
POST /chat	POST	{question, thread_id, job_id}	text/event-stream SSE	Streams tokens word by word
WS /ws/{job_id}	WebSocket	job_id in URL	{event, video_id} pushed	Server pushes when Instagram ready

5.3 main.py

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from routes import ingest, status, chat, ws

app = FastAPI(title='VideoRAG API')

app.add_middleware(
    CORSMiddleware,
    allow_origins=['http://localhost:3000'],
    allow_methods=['*'],
    allow_headers=['*'],
)
```

```
app.include_router(ingest.router)
app.include_router(status.router)
app.include_router(chat.router)
app.include_router(ws.router)
```

5.4 POST /ingest

```
# routes/ingest.py
from fastapi import APIRouter, BackgroundTasks
from services.ingestion import run_ingestion_pipeline
from state import set_job_state
import uuid

router = APIRouter()

@router.post('/ingest')
async def ingest(req: IngestRequest, background_tasks: BackgroundTasks):
    job_id = str(uuid.uuid4())
    await set_job_state(job_id, {'pct': 0, 'stage': 'queued', 'video_ready': []})
    background_tasks.add_task(run_ingestion_pipeline, req.yt_url, req.ig_url, job_id)
    return {'job_id': job_id, 'status': 'queued'}
```

5.5 POST /chat — SSE Stream

```
# routes/chat.py
from fastapi import APIRouter
from fastapi.responses import StreamingResponse
from services.llm import rag_graph
import json

router = APIRouter()

@router.post('/chat')
async def chat(req: ChatRequest):
    async def token_stream():
        async for token in rag_graph.astream(
            {'question': req.question, 'job_id': req.job_id},
            config={'configurable': {'thread_id': req.thread_id}},
            stream_mode='messages'
        ):
            if token and hasattr(token[0], 'content'):
                yield f'data: {token[0].content}\n\n'
            yield 'data: [DONE]\n\n'

    return StreamingResponse(
        token_stream(),
        media_type='text/event-stream',
        headers={'Cache-Control': 'no-cache', 'X-Accel-Buffering': 'no'}
    )

# CRITICAL: X-Accel-Buffering: no prevents nginx from buffering
# Without it, user sees nothing until the full response is complete
```


Chapter 6 — Next.js Frontend

6.1 File Structure

```
frontend/
├── app/
│   ├── page.tsx           # main 3-column layout
│   ├── layout.tsx        # root layout
│   └── globals.css       # Tailwind base
├── components/
│   ├── VideoCard.tsx     # URL input + thumbnail + stats
│   ├── ChatPanel.tsx     # messages + input + suggested Qs
│   ├── MessageBubble.tsx # renders citation badges inline
│   ├── ProgressBar.tsx  # ingestion progress with stages
│   └── StatusBadge.tsx   # ready / loading / error
├── hooks/
│   ├── useIngest.ts      # POST /ingest + poll /status
│   ├── useChat.ts        # SSE stream consumer
│   └── useWebSocket.ts   # /ws push listener for 'ready' events
├── lib/
│   ├── api.ts            # typed fetch wrappers
│   └── types.ts          # shared TypeScript interfaces
```

6.2 Key React Hook — useChat (SSE Consumer)

```
// hooks/useChat.ts
export function useChat(threadId: string) {
  const [messages, setMessages] = useState<Message[]>([])
  const [streaming, setStreaming] = useState(false)

  const sendMessage = async (question: string) => {
    setStreaming(true)
    let buffer = ''
    // Optimistic UI: add user message immediately
    setMessages(prev => [...prev,
      { role: 'user', content: question },
      { role: 'assistant', content: '' }
    ])
    const res = await fetch('/api/chat', {
      method: 'POST',
      body: JSON.stringify({ question, thread_id: threadId }),
      headers: { 'Content-Type': 'application/json' }
    })
    const reader = res.body!.getReader()
    const decoder = new TextDecoder()
    while (true) {
      const { done, value } = await reader.read()
      if (done) break
      const chunk = decoder.decode(value)
      for (const line of chunk.split('\n')) {
        if (!line.startsWith('data: ')) continue
        const data = line.slice(6)
        if (data === '[DONE]') { setStreaming(false); continue }
        buffer += data
        // Append each token to last message
      }
    }
  }
}
```

```

    setMessages(prev => {
      const updated = [...prev]
      updated[updated.length - 1].content = buffer
      return updated
    })
  }
}
return { messages, streaming, sendMessage }
}

```

6.3 Citation Badge Rendering

The LLM is prompted to write [Video A, chunk 3] inline. MessageBubble.tsx parses these and renders them as coloured badges, not plain text.

```

// components/MessageBubble.tsx
function renderWithCitations(text: string) {
  const pattern = /(\[Video [AB], chunk \d+\]|\[metadata\])/g
  const parts = text.split(pattern)
  return parts.map((part, i) => {
    if (pattern.test(part)) {
      return <CiteBadge key={i} label={part} />
    }
    return <span key={i}>{part}</span>
  })
}

function CiteBadge({ label }: { label: string }) {
  const color = label.includes('Video A') ? 'bg-teal-100 text-teal-800' : 'bg-coral-100 text-coral-800'
  return <span className={`inline-block text-xs px-1.5 py-0.5 rounded mx-0.5 ${color}`}>{label}</span>
}

```

Chapter 7 — Timing & Performance Analysis

7.1 Ingestion Timing — 1-Minute Video (Whisper API)

Step	YouTube	Instagram	Parallel?
Transcript fetch	~0.3s (transcript-api)	~3-5s (Whisper API)	Both fire simultaneously
Metadata fetch	~0.3s (Data API v3)	~0.5s (instaloader)	Runs in same gather()
Engagement rate compute	<0.01s	<0.01s	Instant, pure math
Chunking (both videos)	~0.05s	~0.05s	Both in parallel threads
BGE-small embedding (~14 chunks)	~0.5s CPU	~0.5s CPU	Both embed in parallel
Qdrant upsert	~0.2s	~0.2s	Both upsert in parallel

Scenario	Time	User experience
YouTube only, sequential	~1.1s	Chat ready in ~1.5s total
Both videos, parallel	~5-6s total (bottleneck = Whisper)	Video A ready in ~1.5s, Video B auto-unlocks after ~5s via WebSocket push
Both videos, CPU Whisper (no API)	~50s total	Video A ready immediately, Video B ready after 50s

7.2 Chat Turn Latency

Step	Time
Load memory from Redis	~1-2ms
Embed question (BGE-small CPU)	~2-3ms
Qdrant search A + B (parallel)	~8-12ms total
Build prompt	<1ms
LLM first token (Groq TTFT)	~200-400ms
Full response (streaming)	1-3s depending on length
Save memory to Redis	~1ms (async, after response starts)

User perception

Because streaming starts at TTFT (~300ms), the user sees the first word appear almost instantly. The full 2-3s answer streams in word by word. This feels significantly faster than waiting 2-3s for a complete response.

Chapter 8 — Scalability: 10,000 Concurrent Users

8.1 What Breaks First and Why

Component	Single-node limit	What breaks	Fix
FastAPI (1 worker)	~200 concurrent SSE streams	New requests queue, latency spikes	Run 8 Uvicorn workers
BGE-small (CPU, 1 thread)	~500 embeddings/min on CPU	Embedding becomes bottleneck	ThreadPoolExecutor + model singleton
Qdrant (single node)	~10k QPS search	Acceptable for 10k users	Qdrant cluster if >10k QPS
Redis (single node)	~100k ops/sec	Not a bottleneck at this scale	Redis Cluster if needed
LangGraph MemorySaver	In-memory, lost on restart	Worker restart = all sessions lost	AsyncRedisSaver (already in our plan)
Groq free tier	14,400 req/day	Rate limit errors at 10/req/user/day	Groq paid tier or multiple keys
Whisper API (OpenAI)	~50 concurrent requests	429 rate limit errors	Queue with exponential backoff

8.2 Token Budget Per User at Scale

Understanding the token cost per chat turn is essential for predicting costs as users scale.

Item	Tokens	Calculation
System prompt	300	Fixed
Conversation history (6 turns)	1,200	~200 tokens/turn x 6
Video A retrieved chunks (4)	480	~120 tokens/chunk x 4
Video B retrieved chunks (4)	480	~120 tokens/chunk x 4
User question	30	Average question length
Total input tokens	2,490	Sum of above
LLM output tokens	500	Typical answer length

Total tokens per turn	2,990	~3,000 rounded
Cost per turn (Groq paid)	\$0.00089	2990 tokens x \$0.59/1M input + \$0.79/1M output
Cost for 10 turns/user/day	\$0.009	Typical session length
Cost for 1000 users/day x 10 turns	\$9/day	\$270/month at Groq paid

8.3 Production Deployment for 10k Users

Infrastructure stack

```
# Process level (handles ~2000-4000 concurrent SSE connections)
gunicorn main:app \
  -k uvicorn.workers.UvicornWorker \
  -w 8 \
  --bind 0.0.0.0:8000 \
  --timeout 300 # SSE connections are long-lived

# Qdrant cluster (3 nodes for HA + distributed search)
docker-compose up qdrant-node-1 qdrant-node-2 qdrant-node-3

# Redis Sentinel (HA memory + pub/sub)
redis-sentinel sentinel.conf

# nginx load balancer
upstream videorag {
  least_conn;
  server 127.0.0.1:8000;
  server 127.0.0.1:8001;
  server 127.0.0.1:8002;
}
# Critical: disable proxy buffering for SSE
proxy_buffering off;
```

8.4 Caching Strategy — Avoiding Repeated Work

What to cache	Cache key	TTL	Saving
Transcript for a URL	sha256(url)	7 days	Never re-fetch same video
Embeddings for a URL	sha256(url)	7 days	Never re-embed same video
Qdrant search results	sha256(question + video_ids)	5 minutes	Same question = instant reply
YouTube metadata	video_id	1 hour	Avoids API quota usage

Deduplication insight

At 1000 creators/day, there is a high probability that popular videos get submitted multiple times. With URL-based caching in Redis, the second submission costs \$0 and takes <100ms instead of 5 seconds. This alone can reduce Whisper API costs by 30-50%.

Chapter 9 — Setup & Installation (Step by Step)

9.1 Prerequisites

- Python 3.11+ (check: `python --version`)
- Node.js 18+ (check: `node --version`)
- Docker Desktop installed and running
- Git installed

9.2 API Keys — How to Get Each One

YouTube Data API v3

13. Go to: <https://console.cloud.google.com>
14. Create a new project (or select existing)
15. Go to APIs & Services > Enable APIs & Services
16. Search for 'YouTube Data API v3' and click Enable
17. Go to APIs & Services > Credentials > Create Credentials > API Key
18. Copy the key — this is your YOUTUBE_API_KEY

OpenAI API (for Whisper)

19. Go to: <https://platform.openai.com/api-keys>
20. Click 'Create new secret key'
21. Copy the key — this is your OPENAI_API_KEY
22. Add billing: <https://platform.openai.com/settings/billing>
23. \$5 credit is enough to run 800+ Instagram reels through Whisper

Groq API (for LLM)

24. Go to: <https://console.groq.com/keys>
25. Sign up (free)
26. Click 'Create API Key'
27. Copy the key — this is your GROQ_API_KEY
28. Free tier: 14,400 requests/day, 6000 tokens/minute — plenty for development

9.3 Docker Setup — Qdrant + Redis

```
# Start Qdrant (vector database)
```

```
docker run -d \
  --name qdrant \
  -p 6333:6333 \
  -v $(pwd)/qdrant_data:/qdrant/storage \
  qdrant/qdrant

# Start Redis (job state + memory)
docker run -d \
  --name redis \
  -p 6379:6379 \
  redis:alpine

# Verify both are running
docker ps

# Test Qdrant
curl http://localhost:6333/health
# Expected: {"title":"qdrant - vector search engine","version":"..."}
```

9.4 Backend Installation

```
# Clone and navigate
cd backend

# Create virtual environment
python -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate

# Install all packages
pip install fastapi uvicorn[standard]
pip install langchain langgraph langchain-groq
pip install qdrant-client sentence-transformers
pip install youtube-transcript-api google-api-python-client
pip install instaloader yt-dlp openai
pip install redis[asyncio] python-dotenv tiktoken
pip install websockets

# Create .env file
cat > .env << EOF
YOUTUBE_API_KEY=your_key_here
OPENAI_API_KEY=your_key_here
GROQ_API_KEY=your_key_here
REDIS_URL=redis://localhost:6379
QDRANT_URL=http://localhost:6333
EOF
```

9.5 Frontend Installation

```
# Navigate to frontend directory
cd frontend

# Install dependencies
npm install

# Create .env.local
```

```
echo 'NEXT_PUBLIC_API_URL=http://localhost:8000' > .env.local

# Install Tailwind (if not already)
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

9.6 Running the Full Stack

```
# Terminal 1: Start backend
cd backend
source venv/bin/activate
uvicorn main:app --reload --port 8000

# Terminal 2: Start frontend
cd frontend
npm run dev

# App available at: http://localhost:3000
# API docs at: http://localhost:8000/docs
# Qdrant dashboard: http://localhost:6333/dashboard
```

Chapter 10 — Execution Plan

The plan is split into two distinct phases. Phase 1 builds the entire backend and validates every step in the terminal before touching any UI. Phase 2 wraps a polished frontend around the proven backend.

Why this order matters

Building backend-first and testing in the terminal means you know your data pipeline, RAG retrieval, and LLM responses work correctly before investing time in UI. Any bugs found in Phase 1 are 10x cheaper to fix than bugs discovered after the UI is built.

Phase 1 — Backend + Terminal Validation

Target: Every step visible in terminal with real data. No frontend required.

Step 1.1 — Docker services (10 minutes)

29. Start Qdrant: `docker run -d --name qdrant -p 6333:6333 qdrant/qdrant`
30. Start Redis: `docker run -d --name redis -p 6379:6379 redis:alpine`
31. Verify: `curl http://localhost:6333/health`
32. Expected terminal output: JSON with qdrant version

Step 1.2 — YouTube ingestion test (20 minutes)

33. Write `services/ingestion.py` with `get_youtube_transcript()` and `get_youtube_metadata()`
34. Write a test script: `python test_youtube.py --url 'https://youtube.com/watch?v=...'`
35. Expected terminal output: full transcript text + metadata dict with `views/likes/engagement_rate`
36. Verify `engagement_rate` is computed correctly by checking math manually

Step 1.3 — Instagram ingestion test (20 minutes)

37. Write `get_instagram_transcript()` with `instaloader` + `Whisper API`
38. Test: `python test_instagram.py --url 'https://instagram.com/reel/...'`
39. Expected: transcript text + metadata dict
40. Check `Whisper API` cost on `OpenAI dashboard` after this test

Step 1.4 — Chunking + embedding test (15 minutes)

41. Write `chunk_transcript()` and `embed_chunks()`
42. Test: `python test_chunks.py --` should print each chunk with index and metadata
43. Verify chunk count: a 1-min video transcript (~600 words) should produce ~12-16 chunks at 500-char size
44. Print one embedding vector to confirm it is `float[384]`

Step 1.5 — Qdrant storage + retrieval test (20 minutes)

45. Write `vector_store.py` with `setup_collection()`, `upsert_chunks()`, `search()`
46. Run full pipeline test: `python test_pipeline.py`
47. The script should ingest one YouTube URL end-to-end and print 'Stored N chunks'
48. Then search: `python test_search.py --query 'what did the creator say about consistency'`
49. Expected: 3-4 chunks with score > 0.7 and correct `video_id` in payload

Step 1.6 — LangGraph RAG test (30 minutes)

50. Write `services/llm.py` with the full graph (`load_memory`, `embed_question`, `retrieve`, `generate`)
51. Test without streaming first: `python test_rag.py --question 'What is the engagement rate of Video A?'`
52. Expected: complete answer citing chunks with engagement rate from metadata
53. Test memory: ask two questions in sequence, verify second answer references first

Step 1.7 — FastAPI routes test (30 minutes)

54. Write all 4 routes: `ingest`, `status`, `chat`, `ws`
55. Start server: `uvicorn main:app --reload`
56. Test POST `/ingest` with `curl` or `httpie`
57. Poll GET `/status/{job_id}` and watch percentage climb in terminal
58. Test POST `/chat` with `curl --no-buffer` (shows SSE streaming in terminal)

```
# Test streaming in terminal
curl -N -X POST http://localhost:8000/chat \
  -H 'Content-Type: application/json' \
  -d '{"question": "Why did Video A perform better?", "thread_id": "test1", "job_id": "..."}'
```

59. You should see tokens appearing word by word in the terminal — this confirms streaming works

Step 1.8 — Full end-to-end terminal test (20 minutes)

60. Write `test_e2e.py` that runs the complete flow from URL to chat answer
61. It should print progress at every step: fetching, chunking, embedding, storing, retrieving, generating
62. Test all 5 question types from the assignment spec

- 63. Verify citations appear in the format [Video A, chunk N]
- 64. Verify memory: ask 3 questions in sequence, confirm the 3rd references context from the 1st

Gate to Phase 2

Do not start Phase 2 until every test in Phase 1 passes with real YouTube and Instagram URLs. The terminal test is your ground truth — if it works here, the frontend is just a display layer.

Phase 2 — Next.js Frontend

Target: wrap the proven backend in a polished, performant UI.

Step 2.1 — Layout and video cards (45 minutes)

- 65. Create 3-column grid layout in app/page.tsx
- 66. Build VideoCard.tsx with URL input, thumbnail placeholder, stats display
- 67. Wire useIngest.ts hook — POST /ingest on URL submit
- 68. Add ProgressBar.tsx that polls /status every 2s
- 69. Add StatusBadge showing 'ready' (green) or 'processing...' (amber)

Step 2.2 — WebSocket for auto-unlock (20 minutes)

- 70. Build useWebSocket.ts that connects to /ws/{job_id}
- 71. When server pushes {event: 'video_ready', video_id: 'B'}, update UI state
- 72. Video B card should auto-unlock and show stats without user refreshing

Step 2.3 — Chat panel (45 minutes)

- 73. Build ChatPanel.tsx with message list and input
- 74. Wire useChat.ts SSE hook — tokens append in real time
- 75. Build MessageBubble.tsx with citation badge parsing
- 76. Add suggested follow-up questions below each AI response

Step 2.4 — Polish and performance (30 minutes)

- 77. Disable chat input while streaming (prevents double-send)
- 78. Add skeleton loading states for video thumbnails
- 79. Show error states when ingestion fails
- 80. Test on mobile viewport — ensure 3-column collapses gracefully

Chapter 11 — Terminal Test Scripts

These scripts let you test the entire pipeline in the terminal before building the frontend. Run them in order.

test_e2e.py — Full pipeline in one script

```
# Run: python test_e2e.py
# This tests the entire flow from URL to chat answer
import asyncio, json
from services.ingestion import ingest_youtube, ingest_instagram
from services.embedder import embed_chunks
from services.vector_store import setup_collection, upsert_chunks, search
from langchain.text_splitter import RecursiveCharacterTextSplitter

YT_URL = 'https://www.youtube.com/watch?v=YOUR_ID_HERE'
IG_URL = 'https://www.instagram.com/reel/YOUR_REEL_HERE/'

async def main():
    print('\n=== STEP 1: Setting up Qdrant collection ===')
    await setup_collection()
    print('Collection ready')

    print('\n=== STEP 2: Ingesting both videos in parallel ===')
    yt_data, ig_data = await asyncio.gather(
        ingest_youtube(YT_URL, video_id='A'),
        ingest_instagram(IG_URL, video_id='B')
    )
    print(f'YouTube: {yt_data["title"]} | Engagement: {yt_data["engagement_rate"]}%')
    print(f'Instagram: @{ig_data["creator"]} | Engagement: {ig_data["engagement_rate"]}%')

    print('\n=== STEP 3: Chunking ===')
    splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=100)
    chunks_a = splitter.create_documents([yt_data['transcript']])
    chunks_b = splitter.create_documents([ig_data['transcript']])
    print(f'Video A: {len(chunks_a)} chunks | Video B: {len(chunks_b)} chunks')

    print('\n=== STEP 4: Embedding ===')
    embedded_a = embed_chunks(['text': c.page_content, 'video_id': 'A', **yt_data} for c
in chunks_a])
    embedded_b = embed_chunks(['text': c.page_content, 'video_id': 'B', **ig_data} for c
in chunks_b])
    print(f'Embeddings shape: {len(embedded_a[0]["vector"])} dimensions')

    print('\n=== STEP 5: Storing in Qdrant ===')
    await asyncio.gather(upsert_chunks(embedded_a), upsert_chunks(embedded_b))
    print(f'Stored {len(embedded_a)} + {len(embedded_b)} chunks')

    print('\n=== STEP 6: Test search ===')
    results = await search('what did the creator say about consistency?', video_id='A',
k=3)
    for r in results:
        print(f' Score: {r.score:.3f} | Chunk: {r.payload["chunk_index"]} | {r.payload["text"][:80]}...')
```

```
print('\n=== ALL STEPS PASSED - Ready for LangGraph RAG test ===')

asyncio.run(main())
```

test_chat.py — Interactive terminal chat

```
# Run: python test_chat.py
# Interactive chat in terminal - tests LangGraph + memory
import asyncio
from services.llm import create_graph

THREAD_ID = 'terminal-test-session-1'
JOB_ID = 'test-job-1'

async def main():
    graph = await create_graph()
    print('VideoRAG terminal chat. Type your question. Ctrl+C to exit.')
    print('Suggested: Why did Video A get more engagement than Video B?')
    print('-' * 60)

    while True:
        question = input('\nYou: ').strip()
        if not question: continue
        print('AI: ', end='', flush=True)
        async for token in graph.astream(
            {'question': question, 'job_id': JOB_ID},
            config={'configurable': {'thread_id': THREAD_ID}},
            stream_mode='messages'
        ):
            if token and hasattr(token[0], 'content'):
                print(token[0].content, end='', flush=True)
        print()

asyncio.run(main())
```

Summary — Why This Stack Wins

Criterion	This Stack	Naive Alternative
Cost at 1000 creators/day	~\$2/day (Groq Whisper Large v3)	\$50-200/day (OpenAI embeddings + GPT-4o)
Time to first token	~300ms (Groq)	1-3s (OpenAI GPT-4o)
Embedding cost	\$0 (BGE local)	\$0.80/day (OpenAI ada-002)
Memory durability	Redis — survives restarts	In-memory — lost on restart
Parallelism	3 parallel stages reduce wait	Sequential = 2x slower
Vector DB cost	\$0 (Qdrant self-hosted)	\$70+/month (Pinecone starter)
Scalability ceiling	10k+ users with 8 workers + cluster	Single process = ~200 concurrent
Vendor lock-in	None — all self-hosted or swappable	High (OpenAI, Pinecone, etc.)